

Bastion host and access control

Follow-along: narrow a site that replies to everybody down to one path in

Overview

The site in this lab has five machines and no access policy. An operator sitting outside it can open a connection to every port on every machine, log in to any of them as **root** with a password, and reach the database server as directly as the web server. Nothing there was configured wrongly; nothing was configured at all, which is the state a set of machines is in the moment they are racked and addressed.

Definition: Bastion host

A machine placed so that it is the only one a network outside the site may open a connection to. Everything else in the site is reachable only from it, so an operator who wants to reach an inner machine logs in to the bastion first and continues from there. In this lab the bastion is 114.1.0.2: after Part 2 the operator's workstation can open a TCP connection to that address on port 22 and to nothing else in the site. It is also called a *jump host*, after what an operator does with it.

You will narrow that site to one path, using three mechanisms that sit in three different places. A packet filter on the edge router decides which connections cross between segments at all. The SSH daemon on each machine decides which source addresses it will authenticate. An entry in an account's **authorized_keys** file decides where one particular key may be used from and what it may run. Each of the three stops something the next one does not, and the lab is arranged so you see what each one leaves behind before you add the one that catches it. The **bastion host** is what the first mechanism creates and the other two are written against.

This lab is about reaching machines over SSH and about the filtering that decides which SSH connections happen. It does not cover the rest of what an administrator hardens on a server, such as which accounts hold privilege, what the services themselves are configured to expose, or how a machine is patched.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's **README.md** and run it again.

The root menu lists one category per row. Select **Host and service management**, then **Bastion host and access control**, to reach this lab's actions:

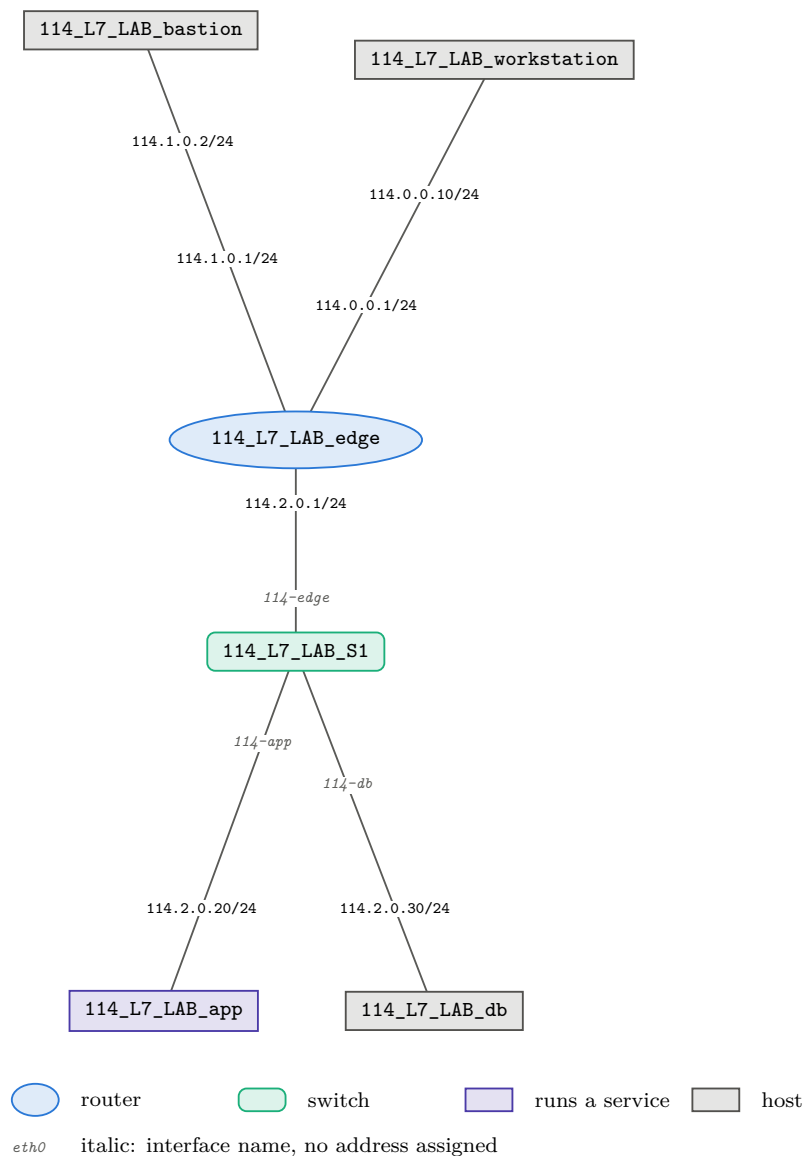
- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before

you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.

- **Status** prints the lab's state and its success oracle, so you read the before and after without shelling in. It sweeps the site from the workstation and reports which ports each machine still replies on. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (workstation, edge, bastion, app, db) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** throws away every setting you have written and returns the lab to its clean Part 1 state, without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Six containers. Two of the three segments hold a single machine each and are point-to-point links to the edge router. The third is switched, and that matters more than it looks: **app** and **db** share one broadcast domain, so a packet from one to the other never reaches the router and no rule you write there can see it.

Machine	Address	Segment	What it runs
114_L7_LAB_workstation	114.0.0.10	external	the operator's client; no server
114_L7_LAB_edge	114.0.0.1	external	router; forwards, filters nothing
	114.1.0.1	DMZ	
	114.2.0.1	inner	
114_L7_LAB_bastion	114.1.0.2	DMZ	sshd; account jump
114_L7_LAB_app	114.2.0.20	inner	sshd and lighttpd on 80; account appops
114_L7_LAB_db	114.2.0.30	inner	sshd; accounts dbops, dbreport
114_L7_LAB_S1	none	inner	Open vSwitch bridge, no STP, no VLANs

Every machine boots with **root** holding the password **Stavanger-4417** and with **sshd** accepting

it. The three service accounts hold passwords too: `jump` has `Kirkenes-9302`, `appops` has `Tromso-6158`, and `dbops` has `Alesund-3720`. The fourth account, `dbreport`, has no password at all and exists for a reporting job you will give a key to in Part 5. Nothing listens on port 443 anywhere, which is deliberate: it gives every sweep in this lab one port whose target is known to be silent.

The workstation carries `nmap` for the sweeps, the OpenSSH client for everything else, and `curl`. The edge router carries `nftables`. You will not need any tool that is not already on the machine the step names.

Assessment

This handout carries eighteen numbered questions, placed at the step that produces the observation each one asks about. Answer all eighteen in a separate document; there is no space to write in here. Several ask you to record a value off your own screen, so answer them as you go rather than at the end, because the next step usually changes the value.

Part 1: the site as delivered

Spawn the lab, then open a shell on the workstation. Every command in this part is typed there.

```
# 114_L7_LAB_workstation: the operator's machine, outside the site
docker exec -it 114_L7_LAB_workstation bash
```

Start by asking what the site replies to. `nmap` sends a TCP SYN to each port named and sorts what comes back into three states.

Definition: open, closed and filtered

Three different things a probed port can do, and `nmap` reports them as three different states. *open* means a listener replied with a SYN-ACK. *closed* means the host replied with a TCP reset: it is reachable and nothing is listening on that port. *filtered* means nothing replied at all within the timeout, which is what a rule that discards the packet looks like from the outside. RFC 9293 is what makes the middle case distinguishable: a TCP endpoint with no listener on the destination port replies to a non-reset segment with a reset rather than staying silent. So `443/tcp closed` on `app` says the packet reached `app` and `app` turned it down; `443/tcp filtered` would say nothing came back at all.

Two of those three states appear in this part; **filtered** is the one Part 2 introduces. Sweep all three machines. `-Pn` tells `nmap` to probe the ports whether or not its own host-discovery probes get a reply, which matters later in the lab and costs nothing now.

```
# 114_L7_LAB_workstation
nmap -Pn -n -p 22,80,443 114.1.0.2 114.2.0.20 114.2.0.30
```

Now use one of the replies. `root` holds a password on every machine and `sshd` accepts it, so the operator can reach an inner machine without touching the bastion at all:

```
# 114_L7_LAB_workstation: password Stavanger-4417
ssh root@114.2.0.20
```

Each machine's `sshd` writes its authentication log to `/var/log/bastion-sshd.log`. In the session you just opened on `app`, read the end of it:

```
# 114_L7_LAB_app: in the SSH session you just opened
tail -3 /var/log/bastion-sshd.log
```

Question 1. Give the state `nmap` reported for port 22 on each of the three machines, and the state it reported for port 443 on `app`. Then give the source address on the `Accepted password for root` line in `app`'s log, and name the machine that address belongs to.

Question 2. Port 443 and port 80 on `db` came back in the same state as each other but that state is not the same one port 80 on `app` came back in. Say what the two silent ports have in common that `app`'s port 80 does not, and what `db` sent back in each case.

The next question is about the shape of the network rather than about any service on it, and the answer is what the whole of Part 5 turns on. Leave the SSH session and run both traces from the machines named.

```
# 114_L7_LAB_workstation: one command, run from your own terminal
docker exec 114_L7_LAB_workstation traceroute -n -q1 114.2.0.30
# 114_L7_LAB_app: the same trace, started from inside the site
docker exec 114_L7_LAB_app traceroute -n -q1 114.2.0.30
```

Question 3. Give the number of hops each trace reported. Then name every device a packet from `app` to `db` passes through, and say what that means for a rule written on `114_L7_LAB_edge`.

Part 2: a default-deny policy at the edge

The site's policy is that the outside may open one connection: to the bastion, on port 22. Every other connection between segments is refused. Writing that down as a filter means starting from refusing everything and naming the exceptions, rather than starting from permitting everything and naming the things to stop. The second way needs a complete list of what is dangerous, which nobody has; the first needs a complete list of what the site does, which the site's operators do have.

All of Part 2 is typed on the edge router.

```
# 114_L7_LAB_edge: the router between the three segments
docker exec -it 114_L7_LAB_edge bash
```

Definition: Hook, base chain and policy

The Linux kernel offers a fixed set of points in the path a packet takes, called hooks. Three matter here: **input** for a packet addressed to this machine, **output** for one this machine generated, and **forward** for one this machine is routing between two of its interfaces. An nftables base chain is a chain attached to one of those hooks, and only a base chain carries a policy: the verdict applied to a packet that reached the end of the chain without any rule accepting or dropping it. The policy is **accept** unless you say otherwise. So **policy drop** on a chain at the **forward** hook means every packet this router would have routed is discarded unless a rule in that chain says to accept it. Until a base chain exists at a hook, nothing at that hook is examined at all, which is the state the edge router boots in.

Create the table and the chain. What you are about to type is a **base chain**: it names the **hook** it attaches to and carries a **policy**. Put no rules in it yet:

```
# 114_L7_LAB_edge
nft add table inet filter
nft -f - <<'EOF'
table inet filter {
    chain forward {
        type filter hook forward priority filter; policy drop;
    }
}
EOF
```

Question 4. Before you look, predict the state a sweep from the workstation will now report for port 22 on the bastion, and for port 80 on app. Then run `docker exec 114_L7_LAB_workstation nmap -Pn -n -p 22,80,443 114.1.0.2 114.2.0.20` from your own terminal and give both states. If either prediction was wrong, say which rule you were assuming existed.

The chain is empty, so its policy is deciding everything. Now write the exceptions. There are three rules and the first is not one of the site's two permitted flows.

Definition: Connection tracking and ct state

Netfilter keeps a table of the connections passing through the machine and labels each packet with where it sits in one. **NEW** is a packet starting a connection the table has not seen. **ESTABLISHED** means the connection has already carried packets in *both* directions. **RELATED** is a packet belonging to a separate connection that an existing one caused, such as an ICMP error about it. The forward hook sees both directions of every conversation, so a chain whose only rules match the client's direction drops each reply on the way back and every permitted connection hangs. One rule matching **ct state established,related** accepts the return traffic for every flow at once, rather than needing a mirror-image rule per flow.

Write all three. The first matches on **connection tracking** state instead of on addresses, so one line covers the reply direction of every flow the other two permit.

```
# 114_L7_LAB_edge: the return traffic for every permitted flow
nft add rule inet filter forward ct state established,related accept
# 114_L7_LAB_edge: the workstation may open SSH to the bastion
nft add rule inet filter forward ip saddr 114.0.0.10 ip daddr 114.1.0.2 tcp dport \
    22 accept
# 114_L7_LAB_edge: the bastion may open SSH to either inner host
nft add rule inet filter forward ip saddr 114.1.0.2 ip daddr 114.2.0.0/24 tcp dport \
    22 accept
```

Read the chain back, and sweep the site again from the workstation:

```
# 114_L7_LAB_edge
nft list ruleset
# 114_L7_LAB_workstation: one command, run from your own terminal
docker exec 114_L7_LAB_workstation nmap -Pn -n -p 22,80,443 114.1.0.2 114.2.0.20 \
    114.2.0.30
```

Question 5. Give the state of port 22 on each of the three machines now, and the state of port 80 on `app`. `lighttpd` on `app` is still running and still serving; say what the state of its port tells the operator running the sweep, and what it does not tell them.

Now take the connection-tracking rule out and see what it was doing. `-a` prints each rule with the handle the kernel gave it, which is what `delete rule` takes.

```
# 114_L7_LAB_edge
nft -a list chain inet filter forward
nft delete rule inet filter forward handle <the handle on the ct state rule>
```

```
# 114_L7_LAB_workstation: one command, run from your own terminal
docker exec 114_L7_LAB_workstation nmap -Pn -n --host-timeout 20s -p 22 114.1.0.2
```

Question 6. Give the state port 22 on the bastion came back in with that rule gone. The rule permitting the workstation to open SSH to the bastion is untouched, so the SYN still crossed the router. Name the packet that did not, and say which of the three rules would have had to match it.

Put the rule back before you go on. It has to sit in the chain, but not necessarily first; rules are evaluated in order and the two accept rules match nothing a return packet would match anyway.

```
# 114_L7_LAB_edge
nft add rule inet filter forward ct state established,related accept
```

Question 7. Apply your three rules to one packet that none of the examples covered: a TCP SYN from `app` at 114.2.0.20 to the workstation at 114.0.0.10, destination port 22. State whether the chain accepts or drops it and name the rule or the policy that decided. Then take a packet travelling the same direction: the SYN-ACK `app` sends back after the bastion opened a connection to 114.2.0.20 on port 22. Say what the chain does with that one, and name the single difference between the two packets that accounts for the two verdicts.

Part 3: reaching an inner machine through the jump host

The site's inner machines are now unreachable from the workstation and the bastion is not. That is the policy, but on its own it is a site the operator cannot administer: the two machines that need administering are exactly the two that stopped replying. The third rule you wrote is what makes the site workable again, and this part is about using it.

Definition: ProxyJump

An OpenSSH client option, spelled `-J` on the command line, that reaches a destination through an intermediate host. The client opens an SSH session to the intermediate host and asks it to connect standard input and output to the destination's address and port, then runs a second, separate SSH session over that channel. The important consequence is who opens which connection: the workstation opens a TCP connection to the bastion, and the *bastion* opens the TCP connection to *app*. Two SSH sessions exist, one inside the other, and the inner one is encrypted end to end between the workstation and *app*, so the bastion carries the bytes without being able to read them.

Reach *appops* on *app* through the bastion with **ProxyJump**. You will be asked for two passwords, the bastion's account first and then *app*'s, because there are two sessions and each one authenticates on its own. You may also be asked to accept two host fingerprints.

```
# 114_L7_LAB_workstation: passwords Kirkenes-9302 then Tromso-6158
docker exec -it 114_L7_LAB_workstation \
  ssh -J jump@114.1.0.2 appops@114.2.0.20
```

You land on *app* as *appops*. Read the login `sshd` just recorded, and compare it with the one from Part 1:

```
# 114_L7_LAB_app: in the session you just opened
grep Accepted /var/log/bastion-sshd.log
```

Question 8. Give the source address on the newest **Accepted password for appops** line and the source address on the **Accepted password for root** line from Part 1. Say which machine each one names, and explain the difference in terms of which machine opened the TCP connection that *app*'s `sshd` accepted.

Question 9. Only one of your three forward-chain rules can have permitted the connection *app* recorded. Name it. Then say what you would have seen instead if you had written that rule with `ip saddr 114.0.0.10` rather than the address it has, and at which point in the `ssh -J` command the failure would have appeared.

Typing the jump host into every command is how it gets left out. Write it into the client's configuration once instead. Leave the SSH session first; this file goes on the workstation.

```
# 114_L7_LAB_workstation: this is output, not commands to type
Host bastion
    HostName 114.1.0.2
    User jump

Host app
    HostName 114.2.0.20
    User appops
    ProxyJump bastion

Host db
    HostName 114.2.0.30
    User dbops
    ProxyJump bastion
```

Put that in `/root/.ssh/config` on the workstation, then check that the short form reaches the same place:

```
# 114_L7_LAB_workstation
docker exec -it 114_L7_LAB_workstation vi /root/.ssh/config
docker exec -it 114_L7_LAB_workstation ssh app
```

Question 10. Every operator session into the site now begins with an authenticated login on the bastion, and `app` no longer sees the workstation's address on any line of its log. Name one fact about who reached which machine that the bastion's log now records and that no machine in this site recorded in Part 1, and name one fact it does not record that `app`'s log still does.

Part 4: one key per account, and no passwords

Everything so far has been reached with a password, and the site has four of them written in this handout. A password is a secret that has to be present at the machine that checks it, so every machine holding one is a place it can be read, guessed, or reused. This part replaces all four with key pairs and then removes the passwords, and it narrows what the jump host is willing to forward while it is at it.

Definition: Public-key authentication and `authorized_keys`

A key pair is two files: a private key the client keeps and a public key that can be handed out freely. The client proves it holds the private key by signing something the server sent; the server never sees the private key and holds nothing worth stealing. The server decides which keys count for an account by reading that account's `~/.ssh/authorized_keys`, one public key per line. So installing `appops`' public key in `/home/appops/.ssh/authorized_keys` on `app` makes that one key open that one account on that one machine, and does nothing anywhere else.

Generate four **key pairs** on the workstation, one per account, so that no key opens more than one **authorized_keys** entry anywhere in the site. the empty argument to `-N` gives each private key an empty passphrase, which keeps this lab's commands short; on a real workstation the private key is the one secret worth protecting with one.

```
# 114_L7_LAB_workstation
docker exec -it 114_L7_LAB_workstation bash
ssh-keygen -t ed25519 -N '' -C jump@workstation -f /root/.ssh/id_jump
ssh-keygen -t ed25519 -N '' -C appops@workstation -f /root/.ssh/id_appops
ssh-keygen -t ed25519 -N '' -C dbops@workstation -f /root/.ssh/id_dbops
ssh-keygen -t ed25519 -N '' -C dbreport@workstation -f /root/.ssh/id_dbreport
```

The bastion goes first, because every other machine is reached through it. `ssh-copy-id` opens an ordinary session, so it asks for the account's password this once:

```
# 114_L7_LAB_workstation: password Kirkenes-9302
ssh-copy-id -i /root/.ssh/id_jump.pub jump@114.1.0.2
```

Name the key in the client configuration so it is used without being asked for. Add one `IdentityFile` line under each `Host` block in `/root/.ssh/config`:

```
# 114_L7_LAB_workstation: this is output, not commands to type
Host bastion
    HostName 114.1.0.2
    User jump
    IdentityFile /root/.ssh/id_jump
```

Add the matching line to the `app` and `db` blocks, pointing at `id_appops` and `id_dbops`. Then install those two keys, each through the jump host. The first contact with `db` asks you to accept its host fingerprint.

```
# 114_L7_LAB_workstation: passwords Tromso-6158 then Alesund-3720
ssh-copy-id -i /root/.ssh/id_appops.pub app
ssh-copy-id -i /root/.ssh/id_dbops.pub db
```

Both accounts should now open with no password at all:

```
# 114_L7_LAB_workstation
ssh app 'id -un; hostname'
ssh db 'id -un; hostname'
```

Now aim one key at the other account:

```
# 114_L7_LAB_workstation
ssh -o IdentitiesOnly=yes -i /root/.ssh/id_appops -o ProxyJump=bastion \
    dbops@114.2.0.30 'id -un'
```

Question 11. Give the message that command printed. Name the file on `114_L7_LAB_db` that `sshd` consulted to reach that decision, and give its full path. Then say what would have to be true of that file for the same command to succeed, and what an attacker who took a copy of `/root/.ssh/id_appops` off the workstation could reach as things stand.

With the keys working, the passwords can go. Edit `sshd_config` on each of the three machines that runs `sshd`. Two facts about that file decide how the edit has to be made. `sshd` uses the first value it obtains for a keyword, so appending a second `PasswordAuthentication` line below

the one already there changes nothing; and `sshd` re-reads the file only when it is sent `SIGHUP`, so an edit with no signal leaves a file saying one thing and a running daemon doing another. Aim the signal with the daemon's pid file: OpenSSH rewrites its own process name, so the listener appears as `sshd: /usr/sbin/sshd -D -e [listener]` and `pkill -x sshd` matches nothing while reporting success.

```
# 114_L7_LAB_bastion: and repeat on 114_L7_LAB_app and 114_L7_LAB_db
docker exec -it 114_L7_LAB_bastion bash
sed -i 's/^PasswordAuthentication yes/PasswordAuthentication no/' \
    /etc/ssh/sshd_config
sed -i 's/^PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
kill -HUP $(cat /run/sshd.pid)
```

Try the password again, and read the whole refusal line rather than only the fact that it failed:

```
# 114_L7_LAB_workstation: password Kirkenes-9302
docker exec -it 114_L7_LAB_workstation \
    ssh -o PubkeyAuthentication=no jump@114.1.0.2
```

The parentheses in that line list the methods the server is still willing to try. Turn off the second one it names, on all three machines, and try once more:

```
# 114_L7_LAB_bastion: and repeat on 114_L7_LAB_app and 114_L7_LAB_db
echo 'KbdInteractiveAuthentication no' >> /etc/ssh/sshd_config
kill -HUP $(cat /run/sshd.pid)
```

Question 12. Give both refusal lines in full, the one before that change and the one after. Say what changed between them, and state what that tells you about the relationship between `PasswordAuthentication` and `KbdInteractiveAuthentication` as settings.

Question 13. You installed each key and only then removed the password. Say what state the bastion would have been left in had you done those two steps in the other order and the key installation then failed, and say why the bastion is the machine in this site where that mistake could not be undone.

One thing on the bastion is still as wide as it was at spawn. The jump host's whole job is to open connections on somebody else's behalf, and nothing yet says which ones.

Definition: AllowTcpForwarding and PermitOpen

An authenticated SSH session may ask the server to open a TCP connection to a third address and carry it over the encrypted channel. `AllowTcpForwarding` decides whether the server will do that at all, and defaults to **yes**, which means any destination the server itself can reach. `PermitOpen` narrows it to a listed set of `host:port` destinations and refuses the rest. On this bastion the list has two entries, `114.2.0.20:22` and `114.2.0.30:22`, so a session on the `jump` account can be forwarded to `app`'s and `db`'s SSH ports and to nothing else, including ports on the bastion itself.

Write the two destinations the site uses into `PermitOpen`:

```
# 114_L7_LAB_bastion
echo 'PermitOpen 114.2.0.20:22 114.2.0.30:22' >> /etc/ssh/sshd_config
kill -HUP $(cat /run/sshd.pid)
```

`ssh -W host:port` is the forwarding request on its own, with no second session on top, so it shows the forward succeeding or failing without anything else in the way. Ask for one destination on the list and one that is not:

```
# 114_L7_LAB_workstation: the first prints a line, the second does not
docker exec 114_L7_LAB_workstation ssh -W 114.2.0.30:22 bastion </dev/null
docker exec 114_L7_LAB_workstation ssh -W 114.2.0.20:80 bastion </dev/null
```

Question 14. Give what each of those two commands printed. The second destination is one the edge router would also have discarded, since no rule of yours permits the bastion to reach port 80 on `app`. Say which of the two devices actually made the decision, and name the observable difference in the two failures that would let you tell those cases apart.

Part 5: restrictions written on the key itself

Three things are now true of the site at once. The edge router permits one connection in and one connection onward from the bastion. Every login is by key, and each key opens one account on one machine. And `app` and `db` sit on a switched segment where nothing either of them sends to the other reaches the router at all. The third fact is the one this part is about: the router is not on that path, so nothing you wrote in Part 2 has any bearing on it, and if a private key ever lands on one of those two machines it is usable from there.

Private keys do land on servers. An operator copies a home directory to a machine to keep working on it, or restores a backup that was taken from a workstation, and the key comes with it. Put both keys where that would put them, and see what each one reaches:

```
# 114_L7_LAB_workstation
docker exec -it 114_L7_LAB_workstation bash
scp /root/.ssh/id_appops db:id_appops.leaked
scp /root/.ssh/id_dbops app:id_dbops.leaked
```

```
# 114_L7_LAB_db: one command, run from your own terminal
docker exec 114_L7_LAB_db ssh -i /home/dbops/id_appops.leaked \
-o StrictHostKeyChecking=no appops@114.2.0.20 'id -un; hostname'
# 114_L7_LAB_app: the same thing in the other direction
docker exec 114_L7_LAB_app ssh -i /home/appops/id_dbops.leaked \
-o StrictHostKeyChecking=no dbops@114.2.0.30 'id -un; hostname'
```

Question 15. State what each of those two commands returned. Then say why no rule you could add to the chain on `114_L7_LAB_edge` would change either result, and give the number of hops from your Part 1 trace that is the evidence for it.

The file that decides which keys open an account can also carry conditions on each key.

Definition: `authorized_keys` options

A line in `authorized_keys` may begin with a comma-separated option list before the key itself, and the options apply to that one key. `from="pattern-list"` restricts the key to connections arriving from a matching source address; a key used from anywhere else is refused even though the signature is correct. `command="..."` replaces whatever the client asked to run. The `no-pty`, `no-port-forwarding`, `no-agent-forwarding` and `no-X11-forwarding` options turn off one facility each. So a line beginning `from="114.1.0.2" ssh-ed25519 AAAA...` accepts that key from the bastion and from no other address, and it is set by whoever owns the account rather than by whoever owns the machine.

Pin `appops`' key with `from=` to the one address the site means it to arrive from. Open a session on `app` through the bastion and edit the file in place; the key body is long, so change only the front of the line.

```
# 114_L7_LAB_workstation
ssh app
```

```
# 114_L7_LAB_app: in the session you just opened
sed -i 's|^ssh-ed25519|from="114.1.0.2" ssh-ed25519|' ~/.ssh/authorized_keys
cut -c1-40 ~/.ssh/authorized_keys
```

There is no daemon to reload: `sshd` reads `authorized_keys` at each authentication attempt, not at startup. Try the copied key from `db` again, then read what `app` recorded about the attempt.

```
# 114_L7_LAB_db: one command, run from your own terminal
docker exec 114_L7_LAB_db ssh -i /home/dbops/id_appops.leaked \
  -o StrictHostKeyChecking=no appops@114.2.0.20 'id -un'
# 114_L7_LAB_app: one command, run from your own terminal
docker exec 114_L7_LAB_app tail -4 /var/log/bastion-sshd.log
```

Question 16. Quote the line `app`'s log wrote about the refusal. It reports two things about the attempt separately; name both, and say which one of the two the operator would have had no way to learn from the client's own error message.

Check that the path the site does use is untouched, and then look at what the same restriction did not do:

```
# 114_L7_LAB_workstation
ssh app 'id -un; hostname'
# 114_L7_LAB_app: the other copied key, still where you put it
docker exec 114_L7_LAB_app ssh -i /home/appops/id_dbops.leaked \
  -o StrictHostKeyChecking=no dbops@114.2.0.30 'id -un; hostname'
```

You could pin `dbops`' key the same way, and for that one key it would work. The reason to reach for something else is that `from=` protects the keys somebody remembered to write it on. An account added next month, or a second key appended to a file that already has a pinned one, carries no restriction at all, and nothing reports that.

Definition: Match in sshd_config

A **Match** line starts a conditional block: every keyword below it applies only to connections meeting its criteria, and the block runs to the next **Match** or to the end of the file, so nothing global may follow one. **Match Address** takes a pattern list of source addresses, read left to right, in which an entry beginning **!** is a negation that wins outright when it matches. **Match Address *,!114.1.0.2** therefore means every source except that one. Because it is a server setting rather than a key setting, it covers every account on the machine, including accounts and keys that do not exist yet.

Add a **Match Address** block to **db**, at the end of the file, and reload:

```
# 114_L7_LAB_db
docker exec -it 114_L7_LAB_db bash
cat >> /etc/ssh/sshd_config <<'EOF'

Match Address *,!114.1.0.2
    PubkeyAuthentication no
    PasswordAuthentication no
EOF
kill -HUP $(cat /run/sshd.pid)
```

sshd -T prints the configuration with every default resolved, and **-C** evaluates the **Match** blocks against a connection you describe rather than one you have to make. Read the same keyword twice, once for each source:

```
# 114_L7_LAB_db
sshd -T -C user=dbops,host=114.1.0.2,addr=114.1.0.2 | grep '^pubkeyauthentication'
sshd -T -C user=dbops,host=114.2.0.20,addr=114.2.0.20 | grep '^pubkeyauthentication'
```

```
# 114_L7_LAB_app: one command, run from your own terminal
docker exec 114_L7_LAB_app ssh -i /home/appops/id_dbops.leaked \
-o StrictHostKeyChecking=no dbops@114.2.0.30 'id -un'
# 114_L7_LAB_workstation: the path the site does use
docker exec 114_L7_LAB_workstation ssh db 'id -un; hostname'
```

Question 17. Give the two values **sshd -T -C** printed and the result of each of the two login attempts. Then name one thing the **Match** block on **db** covers that adding **from="114.1.0.2"** to **dbops'** key would not have covered, and one thing **from=** can express that the **Match** block cannot.

The last account has no key yet. **dbreport** exists for a job that reads one figure off **db** on a schedule, and giving it a key the ordinary way would give whoever holds that key a shell on the machine the site cares most about.

Definition: Forced command and SSH_ORIGINAL_COMMAND

`command=...` in an `authorized_keys` entry replaces whatever the client asked to run, for that key, every time. The client's request is not refused and it is not reported as an error; it is discarded, and the named command runs instead. What the client asked for is put in the environment variable `SSH_ORIGINAL_COMMAND`, which is the only place it survives, so a command that wants to act on the request has to read it there. A key carrying `command="/usr/local/bin/db-report"` runs `db-report` whether the client asked for `uptime`, for a shell, or for nothing at all.

The reporting script is already on `db` at `/usr/local/bin/db-report`. Install `dbreport`'s public key with a **forced command** in front of it, which is what turns the account into that one command:

```
# 114_L7_LAB_workstation: print the key, then paste it into the file on db
docker exec 114_L7_LAB_workstation cat /root/.ssh/id_dbreport.pub
```

```
# 114_L7_LAB_db: with <key> replaced by the line you just printed
docker exec -it 114_L7_LAB_db bash
mkdir -p /home/dbreport/.ssh
OPTS='command="/usr/local/bin/db-report",from="114.1.0.2",no-pty'
OPTS="$OPTS,no-port-forwarding,no-agent-forwarding,no-X11-forwarding"
echo "$OPTS <key>" > /home/dbreport/.ssh/authorized_keys
chown -R dbreport:dbreport /home/dbreport/.ssh
chmod 700 /home/dbreport/.ssh
chmod 600 /home/dbreport/.ssh/authorized_keys
```

Ask it for three different things:

```
# 114_L7_LAB_workstation
docker exec 114_L7_LAB_workstation ssh -i /root/.ssh/id_dbreport \
-o ProxyJump=bastion dbreport@114.2.0.30
docker exec 114_L7_LAB_workstation ssh -i /root/.ssh/id_dbreport \
-o ProxyJump=bastion dbreport@114.2.0.30 uptime
docker exec 114_L7_LAB_workstation ssh -i /root/.ssh/id_dbreport \
-o ProxyJump=bastion dbreport@114.2.0.30 'cat /etc/shadow'
```

Question 18. Give the two lines the third command printed. Say what the second of those two lines is reading, and what would have happened to that same request had the option list on the key been identical except for the `command=` entry.

Checks: what "done" means

Run `Status` and read the oracle at the foot of its output. The site is finished when all of these hold at once:

- The sweep from the workstation reports port 22 on the bastion **open** and every other port on all three machines **filtered**.
- No password opens any account anywhere; the client's refusal names **publickey** and nothing else.

- **appops'** key and **dbops'** key each open their own account through the bastion, and **app** and **db** both record the login as arriving from 114.1.0.2.
- The copy of **appops'** key on **db** is refused by **app**, and the copy of **dbops'** key on **app** is refused by **db**.
- **dbreport** runs the report whatever the client asks for.
- **app's** web service still serves a request made from inside the site.

That last one is not a formality. A configuration that stopped everything would satisfy the five above it and be worth nothing; the site exists to serve, and every restriction here is measured against a path the site still needs.

Safety note

Every container in this lab sits on an isolated Open vSwitch bridge and two point-to-point links with no route off the host. The sweeps, the key copies and the login attempts reach only the five machines this lab spawned, and none of them can leave. Nothing in this lab is aimed at any address outside 114.0.0.0/8.

Teardown

Remove the lab when you are done: pick **Teardown** in the TUI, or run `scripts/teardown.sh` directly. It deletes this lab's six containers and nothing else.